

Jeremy Troy Suchanski
CS 499: Senior Project
Teacher: Rob Tuft
May 30, 2026

Home Gig



Team Members: Jeremy Troy Suchanski

Project Description

Home Gig is a small-business operations platform designed for home-run businesses that manage orders through disconnected workflows such as phone calls, text messages, social media, spreadsheets, and in-person sales. The system centralizes customer orders, inventory, payments, and business records into one simplified operational dashboard.

The initial demonstration use case focuses on a small local farm operation that currently manages orders through disconnected and low-technology manual workflows. The application will help organize and centralize daily business workflows by replacing disconnected manual processes with a single system for managing products, inventory, orders, payments, and reporting. Home Gig emphasizes operational workflow management and bookkeeping support rather than competing directly with large eCommerce platforms.

Why This Matters

- Many home-run businesses rely on manual and disconnected processes to manage orders and business records.
- Managing orders from multiple sources increases the likelihood of missed orders, duplicate data entry, and bookkeeping errors.
- Small business owners often need simple organizational tools rather than large and complex commercial business platforms.

- Centralized workflow management can improve business organization, record keeping, and operational efficiency.

What the Project Will Deliver

1. A secure owner dashboard with authentication and authorization.
 2. Product and inventory management functionality.
 3. Centralized order management and manual order entry workflows.
 4. Payment tracking status management.
 5. Spreadsheet export functionality for bookkeeping and accounting purposes.
 6. A responsive web application with Progressive Web Application (PWA) support for mobile devices.
-
-

Section 1: Introduction – Purpose of Project

The Problem

Many small home-run businesses manage their operations through disconnected and low-technology workflows. Customer orders may be received through phone calls, text messages, social media messages, spreadsheets, handwritten notes, and in-person conversations. Because information is spread across multiple systems, business owners often spend significant time manually organizing orders, tracking inventory, recording payments, and maintaining bookkeeping records.

Current Pain Points

- Customer orders are received from multiple sources and are difficult to track in one location.
- Business owners often rely on spreadsheets, handwritten notes, or memory to manage daily operations.
- Inventory levels can become inaccurate when orders are not consistently recorded.

- Payment tracking and bookkeeping require significant manual effort.
- Managing order status across multiple communication channels can be time-consuming and error-prone.
- Duplicate data entry and missed orders can occur when information is maintained in multiple locations.

Who This Affects

The primary users of the system are owners of small home-run businesses that sell products directly to customers through a combination of phone orders, text messages, social media interactions, and in-person sales. These businesses often lack affordable software solutions designed specifically for their operational needs and instead rely on manual processes to manage products, inventory, orders, payments, and business records.

Why a Software Solution is the Right Fit

Product management, inventory tracking, order management, payment tracking, and spreadsheet reporting are all structured business workflows that can be organized and automated through a centralized software platform. Rather than maintaining information across multiple disconnected systems, business owners can manage their operations from a single dashboard designed specifically for their workflow.

Home Gig addresses this problem by providing a centralized business operations platform that combines product management, inventory tracking, order management, payment tracking, and spreadsheet export functionality into one application. By reducing reliance on disconnected manual processes, the system can improve organization, simplify bookkeeping, and help business owners operate more efficiently.

Project Scope

The initial project scope focuses on providing a functional minimum viable product that supports the core operational workflows of a small home-run business.

Features such as a public storefront, customer online ordering, customer order history, and other customer-facing functionality may be pursued as ultra stretch goals that would significantly expand the scope of the project if development progress allows but are not required for successful project completion.

Section 2a: Must Have Requirements

MH-1: User Authentication

Requirement:

The system shall allow a business owner to securely log into the application and access protected application features only after successful authentication.

Success Measure:

A user can successfully authenticate using valid credentials and gain access to the application dashboard. Protected API endpoints reject unauthenticated requests.

Demonstration:

Log in using a valid account and display the owner dashboard. Attempt to access a protected API endpoint without authentication and demonstrate that access is denied. Then access the same endpoint after authentication and demonstrate successful access.

MH-2: Product Management

Requirement:

The system shall allow a business owner to create, edit, view, and delete products.

Success Measure:

Product information is stored and displayed correctly within the system.

Demonstration:

Create a product, edit the product information, display the updated product details, and delete it.

MH-3: Inventory Management

Requirement:

The system shall allow a business owner to track product inventory quantities.

Success Measure:

Inventory values are stored and updated accurately.

Demonstration:

Adjust inventory quantities and display the updated inventory information.

MH-4: Centralized Order Management

Requirement:

The system shall allow a business owner to create, view, edit, and manage customer orders entered through phone calls, text messages, verbal requests, and in-person sales.

Success Measure:

Orders are successfully stored, updated, and displayed within the order management system.

Demonstration:

Create an order through the order management workflow and display the saved order.

MH-5: Customer Management

Requirement:

The system shall allow a business owner to create, view, edit, and manage customer records.

Success Measure:

Customer information is stored and displayed accurately within the system.

Demonstration:

Create, edit, and display customer records.

MH-6: Payment Tracking

Requirement:

The system shall allow a business owner to record and track customer payments.

Success Measure:

Payment information is stored and displayed accurately.

Demonstration:

Record a payment and display the updated payment status associated with the order.

MH-7: CSV Export

Requirement:

The system shall allow a business owner to export order and payment information into CSV files.

Success Measure:

The system successfully generates and downloads a CSV file containing order and payment information. Exported files shall include metadata identifying the report generation service and the date and time the file was generated.

Demonstration:

Generate and download a CSV export containing order and payment data. Display the exported file and verify that it includes report generation metadata and the date and time the export was created.

MH-8: Go System Status Service

Requirement:

The system shall include a Go-based service that provides status information for the Home Gig application and communicates its operational status to the application through a status endpoint.

Success Measure:

The Go service successfully runs, responds to status requests, and displays an operational status within the Home Gig application's System Services panel.

Demonstration:

Start the Go service and open the Home Gig application. Expand the System Services panel and display the Go Service status as Online, confirming that the service is operational and communicating within the Home Gig environment.

Section 2b: Stretch Requirements

Stretch Requirement Prioritization

The following stretch requirements are listed in priority order. Higher-priority stretch requirements provide the greatest value relative to implementation effort and will be considered before lower-priority enhancements if development time permits.

Priority Level	Stretch Requirements
Highest Priority	ST-1 Dashboard Analytics Summary ST-4 Automated Inventory Adjustments ST-5 Customer Management Dashboard ST-2 Automated Low Inventory Alerts ST-3 Search and Filtering
Medium Priority	ST-6 Excel Export ST-7 Product Import from CSV ST-8 Customer Import from CSV ST-9 Order Import from CSV ST-10 Payment Import from CSV ST-11 Advanced Spreadsheet Reporting
Lower Priority	ST-12 Go Reporting Service ST-13 Self-Service User Registration ST-14 PWA Support

	ST-15 Excel Import ST-16 Delivery Scheduling and Tracking
Future Expansion / Large Scope Enhancements	UST-1 Public Product Catalog UST-2 Online Customer Ordering UST-3 Customer Order History

The Priority Rationale

Highest Priority stretch requirements provide the greatest user value while requiring relatively small changes to the core application architecture. These features improve daily business operations and provide strong demonstration value.

Medium Priority stretch requirements expand data import, export, and reporting capabilities while leveraging existing application functionality.

Lower Priority stretch requirements introduce new technologies, deployment considerations, authentication workflows, and operational processes that require additional development effort and testing.

UST (Ultimate Stretch) requirements represent significant scope expansions that transform Home Gig from an owner-focused business management application into a customer-facing commerce platform. These enhancements are only intended for consideration after all Must Have requirements and higher-priority stretch requirements have been completed.

ST-1: Dashboard Analytics Summary

Requirement:

The system should display business summary metrics on the owner dashboard.

Success Measure:

The dashboard displays calculated values such as total orders, total sales, unpaid orders, and low inventory products.

Demonstration:

Open the dashboard and display summary metrics generated from application data.

ST-2: Automated Low Inventory Alerts

Requirement:

The system should notify the business owner when a product inventory quantity falls below a set threshold.

Success Measure:

The system identifies products below the threshold and displays a low inventory warning.

Demonstration:

Set a product inventory quantity below the threshold and display the low inventory alert.

ST-3: Search & Filtering

Requirement:

The system should allow a business owner to search and filter products, customers, and orders using basic search criteria.

Success Measure:

The system displays matching records based on the user's search input or selected filter criteria.

Demonstration:

Enter a search term or select a filter and display matching products, customers, or orders within the application.

ST-4: Automated Inventory Adjustments

Requirement:

The system should automatically reduce inventory quantities when customer orders are recorded.

Success Measure:

Inventory values are updated automatically based on ordered quantities.

Demonstration:

Create an order and display the resulting inventory reduction.

ST-5: Customer Management Dashboard

Requirement:

The system should provide a dedicated customer management screen.

Success Measure:

The business owner can view, search, edit, and manage customer information.

Demonstration:

Create, edit, and search customer records within the customer management interface.

ST-6: Excel Spreadsheet Export

Requirement:

The system should allow a business owner to export order and payment information into Microsoft Excel-compatible (.xlsx) files.

Success Measure:

The system successfully generates and downloads an Excel-compatible spreadsheet containing order and payment information.

Demonstration:

Generate and download an Excel spreadsheet export containing order and payment data.

ST-7: Product Import from Spreadsheet

Requirement:

The system should allow a business owner to import product information from a CSV file.

Success Measure:

The system successfully reads uploaded product data and creates product records within the application.

Demonstration:

Upload a CSV file containing product information and display the imported products within the product management screen.

ST-8: Customer Import from Spreadsheet

Requirement:

The system should allow a business owner to import customer information from a CSV file.

Success Measure:

The system successfully reads uploaded customer data and creates customer records within the application.

Demonstration:

Upload a CSV file containing customer information and display the imported customer records within the customer management screen.

ST-9: Order Import from Spreadsheet

Requirement:

The system should allow a business owner to import orders information from a CSV file.

Success Measure:

The system successfully reads uploaded order data and creates order records within the application.

Demonstration:

Upload a CSV file containing order information and display the imported orders within the order management screen.

ST-10: Payment Import from Spreadsheet

Requirement:

The system should allow a business owner to import payment information from a CSV file.

Success Measure:

The system successfully reads uploaded payment data and creates payment records within the application.

Demonstration:

Upload a CSV file containing payment information and display the imported payment records within the application.

ST-11: Advanced Spreadsheet Reporting

Requirement:

The system should allow the business owner to export filtered spreadsheet reports based on selected criteria.

Success Measure:

The user can generate reports filtered by date range, payment status, customer, or delivery status.

Demonstration:

Generate and download a filtered spreadsheet report.

ST-12: Go Reporting Service

Requirement:

The system should support report and spreadsheet generation through a dedicated Go-based reporting service and include metadata within generated reports indicating that they were produced by the Go Reporting Service.

Success Measure:

The Go service successfully generates a requested spreadsheet or report, returns the completed file to the application, updates the Spreadsheet Generator status within the System Services panel to Go, and includes metadata identifying the Go Reporting Service as the report generator.

Demonstration:

Generate a spreadsheet or report from the Home Gig application and download the resulting file. Demonstrate that the Go Reporting Service processed the request by displaying Go as the Spreadsheet Generator within the System Services panel and by displaying metadata within the generated report indicating that it was produced by the Go Reporting Service.

ST-13: Self-Service User Registration

Requirement:

The system should allow a new business owner to create an account through a registration screen.

Success Measure:

The system validates registration information, prevents duplicate accounts, enforces password rules, and creates a new user account when valid information is submitted.

Demonstration:

Complete the registration form using valid information, create a new account, and log in with the new credentials. Also demonstrate validation by submitting invalid information or an already-used email address.

ST-14: Progressive Web Application (PWA) Support

Requirement:

The system should provide a responsive user interface and support installation as a Progressive Web Application (PWA).

Success Measure:

The application functions correctly on desktop and mobile devices and can be installed as a PWA on supported devices.

Demonstration:

Display the application on a mobile-sized screen and demonstrate successful PWA installation.

ST-15: Excel Spreadsheet Import

Requirement:

The system should allow a business owner to import business data from Microsoft Excel-compatible (.xlsx) files.

Success Measure:

The system successfully reads uploaded Excel spreadsheet data and creates or updates records within the application.

Demonstration:

Upload an Excel spreadsheet containing business data and display the imported records within the application.

ST-16: Delivery Scheduling and Tracking

Requirement:

The system should allow the business owner to schedule delivery dates and track delivery progress.

Success Measure:

Delivery dates and delivery statuses are stored and displayed within the application.

Demonstration:

Create a delivery schedule and update delivery status information.

Section 2c: Ultra Stretch Requirements

The following requirements represent significant scope expansions beyond the planned minimum viable product and standard stretch goals. These requirements introduce customer-facing functionality and would substantially increase the complexity of the system. They will only be pursued if development progress significantly exceeds expectations and all Must Have and Stretch Requirements have been completed.

UST-1: Public Product Catalog

Requirement:

The system should allow customers to view available products through a public product catalog.

Success Measure:

Customers can access a public product page and view product names, descriptions, prices, and availability.

Demonstration:

Open the public product catalog and display available products without logging in as the business owner.

UST-2: Online Customer Ordering

Requirement:

The system should allow customers to submit orders through a public online ordering form.

Success Measure:

A customer can select products, enter order information, and submit an order that is saved in the owner dashboard.

Demonstration:

Submit a customer order through the public ordering form and display the order in the centralized order management screen.

UST-3: Customer Order History

Requirement:

The system should allow customers to view their previous orders.

Success Measure:

A customer can access a list of previous orders associated with their account or contact information.

Demonstration:

Log in or search as a customer and display previously submitted orders.

Section 3: Overview of the Product

Workflow

Home Gig is designed to provide a centralized operational dashboard for small home-run business owners. The system allows business owners to manage products, inventory, customers, orders, payments, and business records through a single application. Rather than relying on disconnected workflows involving spreadsheets, handwritten notes, text messages, and social media conversations, the owner can manage the entire order lifecycle from one system.

The primary workflow of the application begins when the business owner logs into the system. After authentication, the owner can manage products and inventory, maintain customer records, enter customer orders from multiple sources, track payments, and export business records for bookkeeping purposes. The outputs of each workflow stage become inputs for subsequent stages, creating a complete operational workflow.

Workflow Phase 1: Authentication

Inputs:

- Username
- Password

Process:

- ❖ The business owner enters login credentials.
- ❖ The system validates the credentials.
- ❖ The system grants access to the application.

Output:

- ✓ Authenticated access to the owner dashboard.

Associated Requirements:

- 1) MH-1 User Authentication: The system shall allow a business owner to securely log into the application.
-

Workflow Phase 2: Product and Inventory Management

Inputs:

- Product information
- Inventory quantities

Process:

- ❖ The owner creates, edits, or deletes products.
- ❖ The owner updates inventory quantities.
- ❖ The system stores and maintains product and inventory records.

Output:

- ✓ Updated product catalog and inventory information.

Associated Requirements:

- 1) MH-2 Product Management: The system shall allow a business owner to create, edit, view, and delete products.
 - 2) MH-3 Inventory Management: The system shall allow a business owner to track product inventory quantities.
-

Workflow Phase 3: Customer Management

Inputs:

- Customer information

Process:

- ❖ The owner creates, edits, views, and manages customer records.
- ❖ The system stores and maintains customer information.

Output:

- ✓ Updated customer records available within the system.

Associated Requirements:

- 1) MH-5 Customer Management: The system shall allow a business owner to create, view, edit, and manage customer records.
-

Workflow Phase 4: Order Management

Inputs:

- Customer information
- Product selections
- Order quantities

Process:

- ❖ The owner receives an order through a phone call, text message, verbal request, or in-person interaction.
- ❖ The owner enters the order into the system.
- ❖ The system associates the order with a customer record.
- ❖ The system stores and tracks the order.

Output:

- ✓ Customer order record available within the centralized order management system.

Associated Requirements:

- 1) MH-4 Centralized Order Management: The system shall allow a business owner to create, view, edit, and manage customer orders entered through phone calls, text messages, verbal requests, and in-person sales.
-

Workflow Phase 5: Payment Tracking

Inputs:

- Customer payment information
- Order information

Process:

- ❖ The owner records payments received from customers.
- ❖ The system associates payment information with the appropriate customer order.

❖ The system updates the payment status of the order.

Output:

✓ Updated payment records associated with customer orders.

Associated Requirements:

1) MH-6 Payment Tracking: The system shall allow a business owner to record and track customer payments.

Workflow Phase 6: Reporting and Export

Inputs:

- Order records
- Payment records

Process:

- ❖ The owner selects data to export.
- ❖ The system generates a CSV file.
- ❖ The file is prepared for download.

Output:

✓ CSV export file for bookkeeping and accounting purposes.

Associated Requirements:

1) MH-7 CSV Export: The system shall allow a business owner to export order and payment information into CSV files.

Resources

Home Gig will use a multi-tier client/server architecture. The application will include an Angular frontend, a Node.js/Express backend API, and a MongoDB database. The frontend will communicate with the backend through REST API requests, and the backend will store and retrieve application data from MongoDB. This structure separates the user interface, business logic, and data storage into distinct layers.

Frontend Tier

The frontend tier will be built using Angular, TypeScript, HTML, CSS, and Bootstrap. Angular will provide the structure for the user interface, routing, components, forms, and communication with the backend API. Bootstrap will be used to support responsive layout and mobile-friendly design. The frontend will provide the owner dashboard screens for authentication, product management, inventory management, customer management, order management, payment tracking, and CSV export access.

Backend Tier

The backend tier will primarily be built using Node.js and Express.js. Express will provide REST API endpoints that allow the frontend to request, create, update, and delete application data. The Node.js/Express backend will handle core business logic such as authentication, product management, inventory management, customer management, order management, payment tracking, data validation, security, and communication with the database. Middleware may be used for authentication, validation, security, and request handling.

To satisfy the senior project requirement to learn and implement a new technology, the project will also include a small Go-based service. At minimum, the Go service will provide status information that allows the Home Gig application to verify that the service is operational and communicating successfully with the Node.js/Express backend. This information may be displayed within the application's System Services panel.

The Go service may also be expanded into a dedicated reporting and spreadsheet-generation service as part of Stretch Requirement ST-12: Go Reporting Service. If implemented, the Node.js/Express backend may communicate with the Go service to generate spreadsheets and reports for download by the user. Generated reports will include metadata identifying the report generation service and the date and time the report was generated.

To reduce project risk, the Node.js/Express backend will also support direct spreadsheet generation. If the Go Reporting Service is not implemented, spreadsheet export functionality will continue to operate through the Node.js/Express backend. This approach allows the project to incorporate a new programming language while maintaining a manageable scope and ensuring that core application functionality remains independent of optional Go-based enhancements.

Go Service

The Go service is included to satisfy the senior project requirement to learn and implement a new technology. The Go System Status Service represents the minimum required implementation of Go within the Home Gig application. At minimum, the service will provide a system status endpoint that confirms the service is running and can communicate with the Node.js/Express backend. This functionality allows the project to demonstrate successful implementation of Go while maintaining a low level of project risk.

Additional Go-based functionality may be implemented through Stretch Requirement ST-12: Go Reporting Service. If development progress and project constraints permit its implementation, the Go service may be expanded into a dedicated reporting and file-generation service. In this role, the Go service would be responsible for processing business data and generating CSV exports and reports for the Home Gig platform.

The primary application functionality will be provided through the Angular frontend, Node.js/Express backend, and MongoDB database. If the Go Reporting Service is implemented, when report generation is requested, the Node.js backend will communicate with the Go service, which will retrieve or receive the required data, generate the requested report, and return the completed file for download. Generated reports will include metadata identifying the report generation service and the date and time the report was generated.

This approach allows the project to incorporate a new programming language and backend technology while maintaining a manageable project scope and clear separation of responsibilities between application management and reporting functionality.

To reduce project risk, the Node.js/Express backend will also support direct CSV generation. If the Go Reporting Service is unavailable or not fully implemented, CSV export functionality will be generated directly by the Node.js backend. This ensures that MH-7 CSV Export remains fully functional while allowing the Go Reporting Service to be implemented as an optional stretch enhancement rather than a dependency.

Database Tier

MongoDB will be used as the database for storing application data. Mongoose will be used as the object modeling library to define schemas and interact with MongoDB from the Node.js backend. The main database collections are expected to include users, products, customers, orders, and payments. Inventory quantities will be stored as part of the product records to keep the MVP scope manageable.

Third-Party Libraries and Packages

The project will use third-party tools and libraries where appropriate instead of building every feature from scratch. Angular, Bootstrap, Node.js, Express.js, MongoDB, and Mongoose will provide the main application framework and data layer. Git and GitHub will be used for version control and repository management. CSV export functionality will be implemented through backend file generation ~~using Node.js packages or built-in~~ JavaScript/Node.js capabilities. If stretch goals are implemented, ExcelJS or SheetJS may be used for Excel-compatible export or import functionality.

Development and Delivery Tools

Cursor and Visual Studio Code will be used as the primary development environments for the project. I will be using Cursor as an AI-assisted development environment that provides code generation, code explanation, debugging

assistance, refactoring support, and project-wide code analysis. Visual Studio Code will continue to be used for development, testing, and integration activities as needed.

Git and GitHub will be used for source code management, version control, and project backup. The application will initially be developed and tested locally on a Windows development machine. If deployment is completed, the frontend and backend may be hosted using a free or low-cost hosting platform, while MongoDB may be hosted locally for development or through a cloud-hosted MongoDB service.

Deployment Considerations

The Home Gig application will be developed and tested within a local development environment. For production deployment and demonstration purposes, the current planned hosting platform is Render, which provides support for Angular and Node.js applications, automatic HTTPS configuration, and GitHub-based deployment workflows.

A custom domain name is not required for successful completion of the project. The application may be accessed using the hosting provider's assigned URL. A custom domain may be added in the future if desired.

AI-Assisted Development Tools

AI-assisted development tools including Cursor, ChatGPT, Codex, GitHub Copilot, Grok, Claude and AI agent-based workflow tools will be used to support research, learning, troubleshooting, debugging, architectural planning, documentation, code generation, code , and development workflow assistance throughout the project.

Cursor will be used to explore AI-assisted software development workflows and evaluate how AI-powered development environments can improve productivity when building full-stack applications. AI agents and agent-based workflow tools may also be used to assist with multi-step development tasks such as planning features, generating documentation, reviewing code structure, identifying bugs, and organizing implementation steps. These tools will be used as development

support resources while ensuring that I understand, test, and can explain the implemented code, architecture, and design decisions.

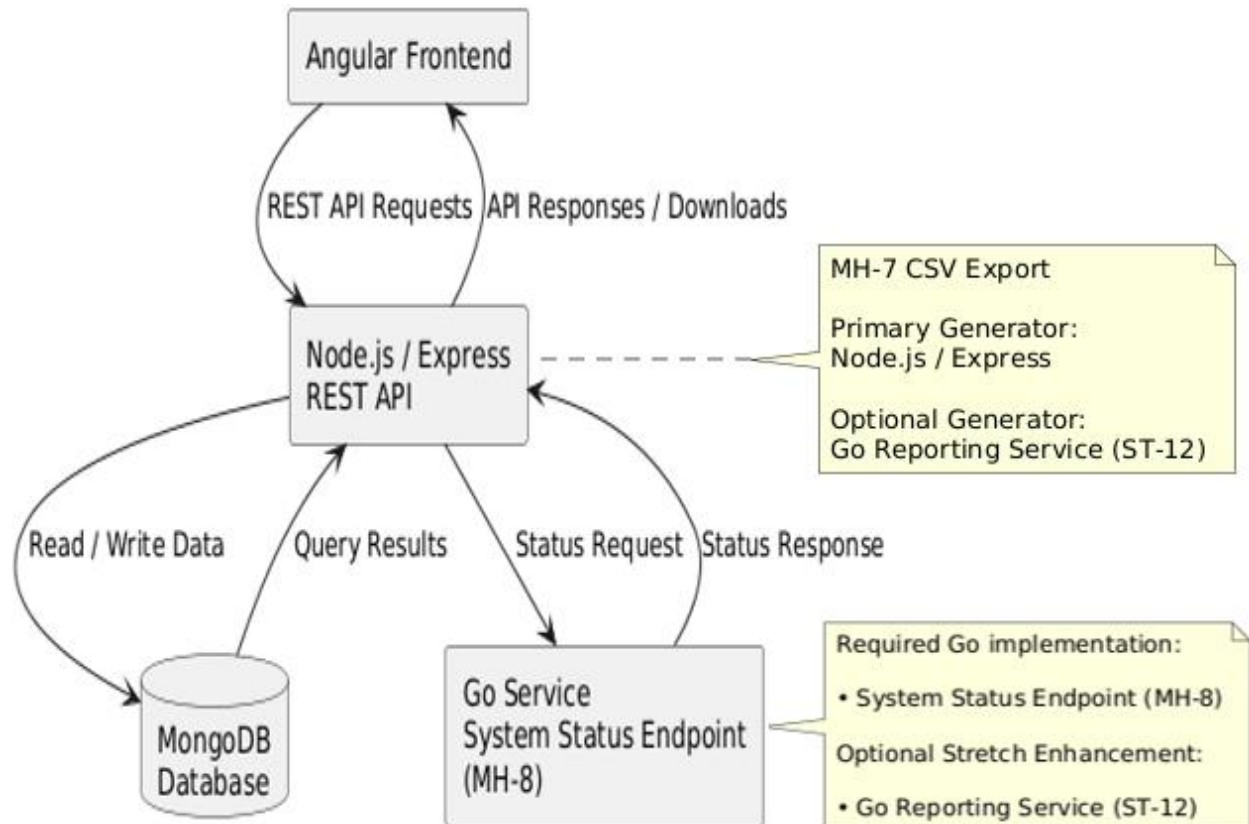
How the Parts Interconnect

The business owner will interact with the Angular frontend through a web browser. When the owner logs in, manages products, updates inventory, creates customers, enters orders, records payments, or exports CSV files, the frontend will send REST API requests to the Node.js/Express backend. The backend will process the request, apply the necessary business logic, interact with MongoDB through Mongoose, and return the appropriate response to the frontend. For CSV export, the backend will retrieve order and payment data, generate a CSV file, and return it to the frontend for download.

Overall Architecture

The system will use a multi-tier client/server architecture:

Home Gig Overall Data Flow Summary



The system will use a multi-tier client/server architecture consisting of an Angular frontend, a Node.js/Express application server, a MongoDB database, and a Go service. The Angular frontend will provide the user interface, the Node.js/Express backend will manage business logic, API communication, authentication, inventory management, customer management, order management, payment tracking, and report generation. MongoDB will provide persistent data storage.

To satisfy the senior project requirement to learn and implement a new technology, the architecture will also include a small Go service. At minimum, the Go service will provide system status information that allows the application to verify service availability and communication with the Node.js/Express backend. Additional functionality may be implemented through Stretch Requirement ST-12: Go Reporting Service. If implemented, the Go service may provide spreadsheet-generation and reporting capabilities while the Node.js/Express backend continues to support direct spreadsheet generation.

This architecture supports separation of concerns, easier debugging, independent service development, reduced project risk, and future expansion. Stretch goals such as Progressive Web Application support, Excel export, spreadsheet import, advanced reporting, or a public storefront can be added later without changing the core structure of the application.

Data at Rest

The Home Gig system will store all persistent data in a MongoDB database. MongoDB was selected because it integrates well with the Node.js and Express backend technology stack, supports flexible document-based data structures, and is appropriate for small business management applications.

User account information will be stored in MongoDB collections. This includes user profile data such as account credentials, name, contact information, business information, and application preferences. User credentials will be stored securely using established authentication and password protection practices rather than being stored in plain text.

Private user and customer information will be protected using appropriate encryption and security practices. Personally identifiable information such as names, phone numbers, email addresses, addresses, and other sensitive business or customer information will be encrypted before being stored within the database. Passwords will not be encrypted directly; instead, they will be stored using secure one-way password hashing techniques. These measures help protect sensitive information from unauthorized access while maintaining application functionality.

The system will also store operational business data generated by users. This includes product information, customer records, inventory quantities, orders, payments, and other business management information entered through the application. If Stretch Requirement ST-16 is implemented, additional delivery scheduling and tracking information may also be stored. The system will organize application data into logical collections to support efficient storage, retrieval, and reporting of business information.

In addition to operational data, the system will generate CSV export files for reporting and accounting purposes. CSV export functionality will be provided directly by the Node.js/Express backend and will generate files on demand using data stored within MongoDB. Generated files will be provided to the user for download but will not serve as the primary data store. MongoDB will remain the system of record for all application data.

As part of Stretch Requirement ST-12: Go Reporting Service, report-generation and file-processing functionality may be delegated to a dedicated Go service. If implemented, the Go Reporting Service will retrieve or receive the required business data, generate the requested report or spreadsheet, and return the completed file to the application for download. Generated reports will include metadata identifying the report generation service and the date and time the report was generated.

If implemented as a stretch goal, additional spreadsheet formats such as Microsoft Excel-compatible (.xlsx) files may also be supported. However, Excel export

functionality is optional and will not be required for successful completion of the project.

Data Modeling Decisions

The Home Gig MVP will use several early data modeling decisions to reduce implementation risk and preserve data consistency between products, inventory, orders, and payments.

Product Deletion

Products will use a soft-delete approach. When a product is deleted, the system will mark the product as inactive rather than permanently removing it from MongoDB. This allows historical orders to continue referencing products that were available when the order was created.

Order Line-Item Snapshots

Order records will store a snapshot of product information at the time the order is created. Each order line item will include the product ID, product name, quantity, unit price, and line total. This ensures that historical orders remain accurate even if a product name or price changes later.

Inventory Adjustments

For the MVP, inventory quantities will be updated manually by the business owner. Automatic inventory reduction when orders are created or edited is included as Stretch Requirement ST-4 and is not required for successful MVP completion.

Payment Model

For the MVP, payment tracking will use a simple payment status model with values such as Paid and Unpaid. Partial payments, refunds, and advanced accounting workflows are outside the MVP scope and may be considered future enhancements.

Data on the Wire

Data will be transmitted between the Angular frontend, the Node.js/Express backend, and the MongoDB database using a multi-tier client/server architecture. The optional Go Reporting Service may also participate in data-processing and report-generation activities if implemented. When a user performs an action such as logging in, managing products, updating inventory, maintaining customer records, creating orders, recording payments, or generating reports, the Angular frontend will send requests to REST API endpoints exposed by the Express backend.

Business logic and data validation will primarily occur on the Node.js/Express backend. After validating the request, the backend will create, read, update, or delete records in MongoDB and return the results to the frontend as JSON data. This approach centralizes business rules and helps ensure data consistency, maintainability, and security.

All communication between application components will use appropriate network security practices. During development, communication may occur within a local development environment. For production deployment, HTTPS should be used between the Angular frontend and backend services to protect sensitive information while in transit. The system will require users to authenticate before accessing protected application features. Authentication and authorization logic will be managed by the Express backend.

Report generation and file-processing operations may be performed by the optional Go Reporting Service if Stretch Requirement ST-12 is implemented. When a user requests a CSV export or report, the Express backend will determine whether report generation should be handled directly by Node.js/Express or delegated to the Go Reporting Service. If delegated, the Express backend will communicate with the Go service through internal API calls. The Go service will receive or retrieve the required data, generate the requested file or report, and return the completed output to the Express backend. The backend will then transmit the generated file to the user's browser for download. If the Go Reporting Service is not implemented or unavailable, the Express backend will generate the

CSV export directly. In either case, generated reports will include metadata identifying the report generation service and the date and time the report was generated.

MongoDB will serve as the authoritative source of business data and will remain the system of record for all application information. Generated CSV files and reports will be temporary outputs created from the stored data rather than permanent records within the system. This approach ensures that all operational information remains centralized within the database while allowing reports to be generated on demand.

Data State

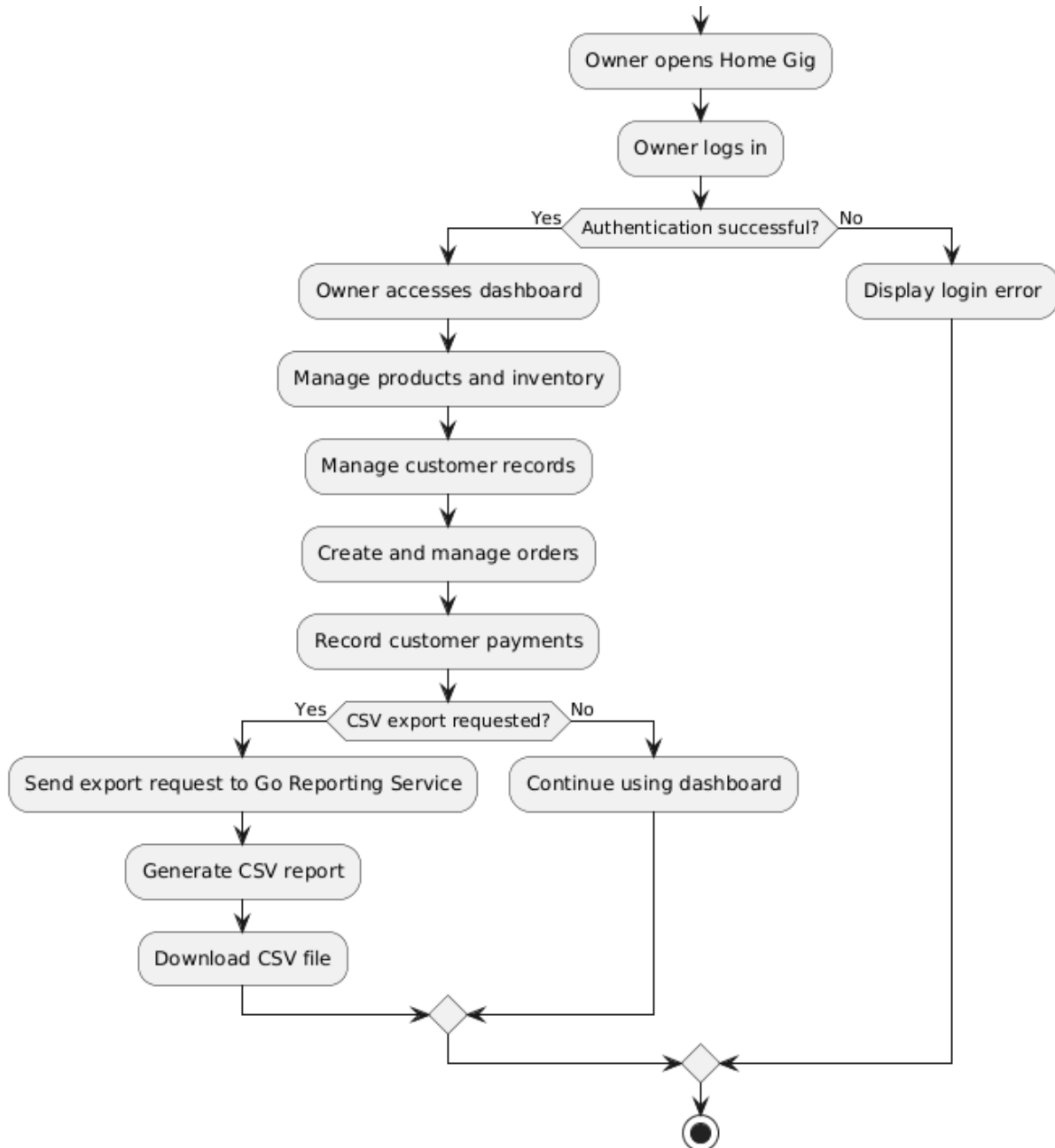
The following Activity Diagrams illustrate how data moves through the Home Gig system during each major workflow phase. Each diagram corresponds to a workflow phase described earlier and identifies where user input originates, where processing occurs, where data is stored, and what output is returned to the user. Together, these diagrams provide a visual representation of the flow of information throughout the system.

Home Gig Workflow Activity Diagrams

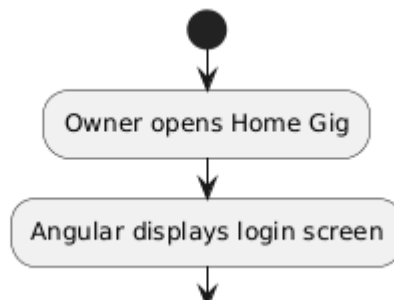
The first Activity Diagram provides a simplified summary of the overall Home Gig data flow. The diagrams that follow break this overall flow into the six major workflow phases described earlier.

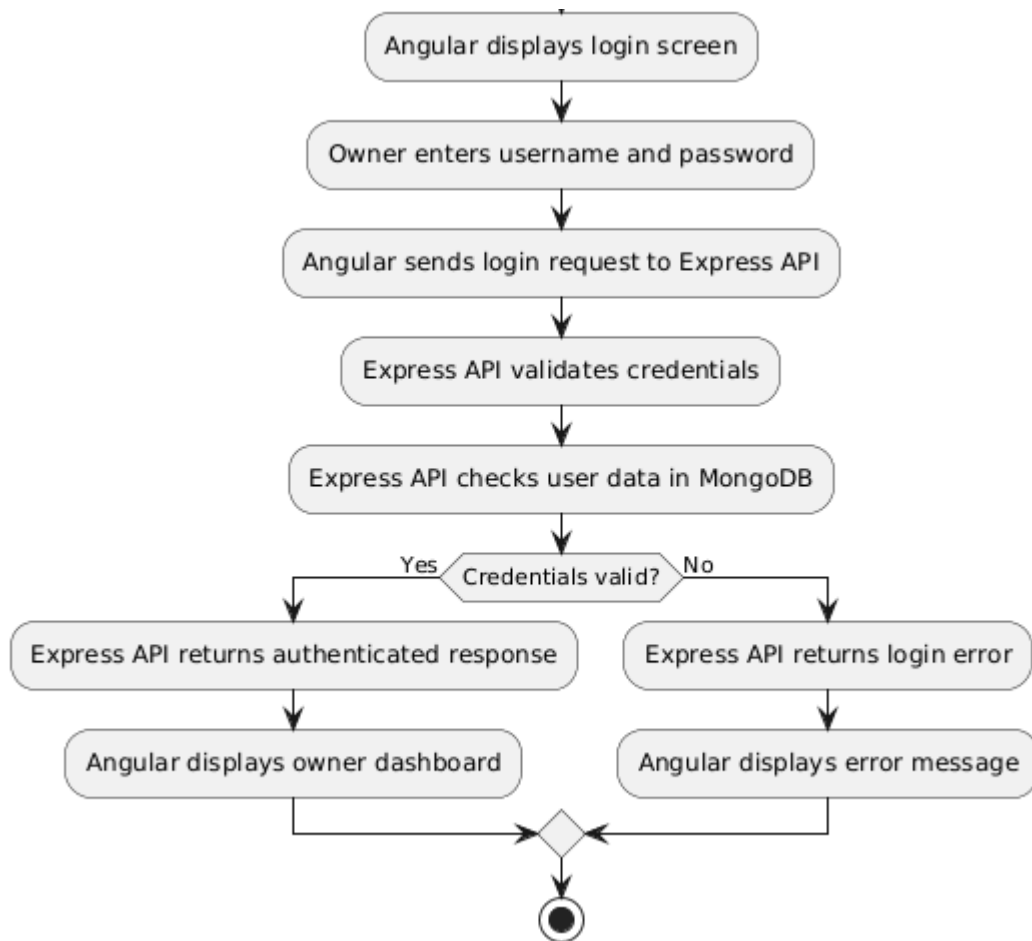
Home Gig Overall Data Flow Summary



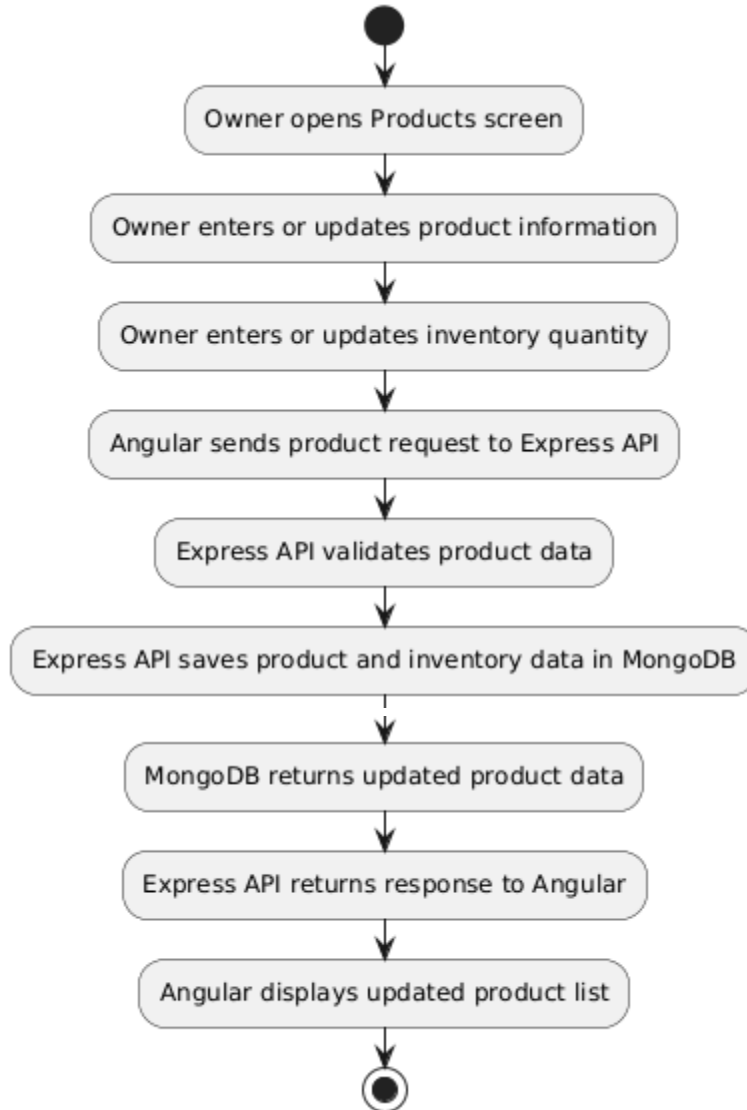


Workflow Phase 1 - Authentication

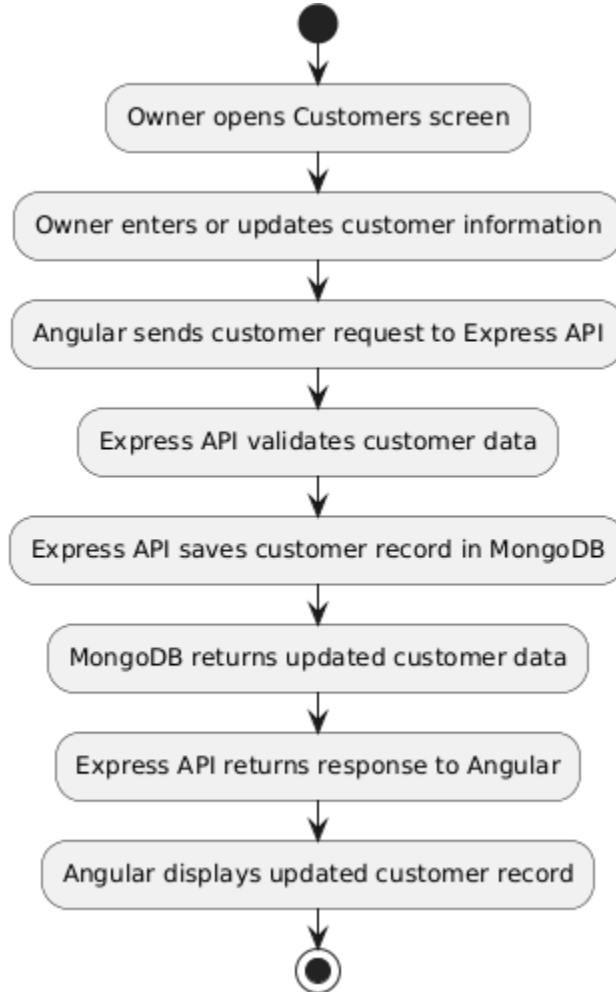




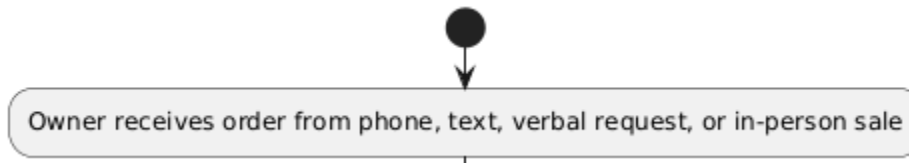
Workflow Phase 2 - Product and Inventory Management

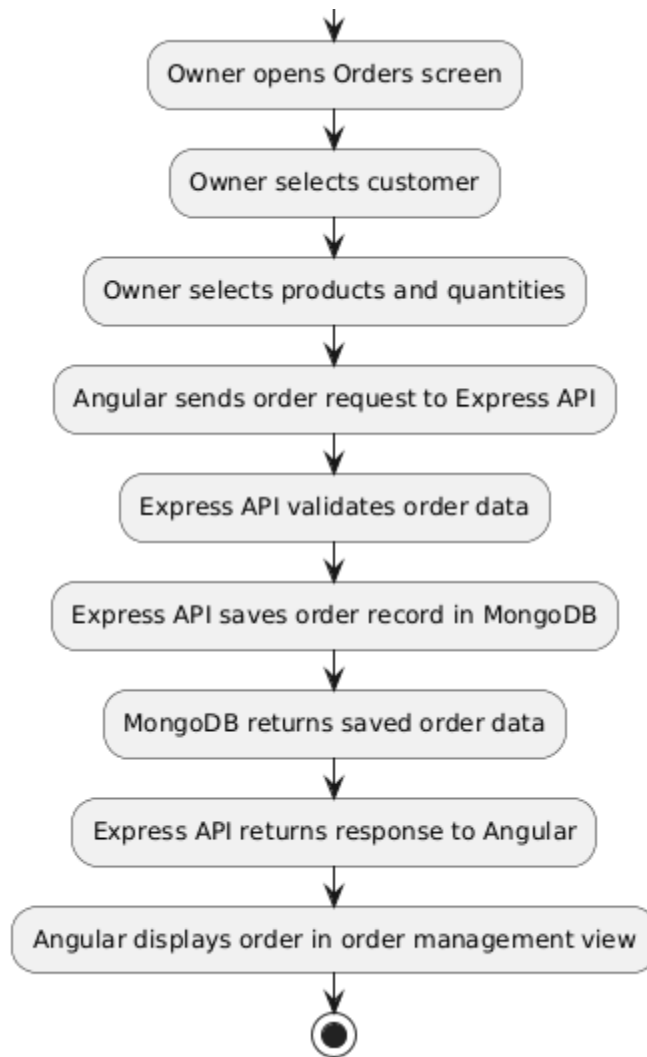


Workflow Phase 3 - Customer Management

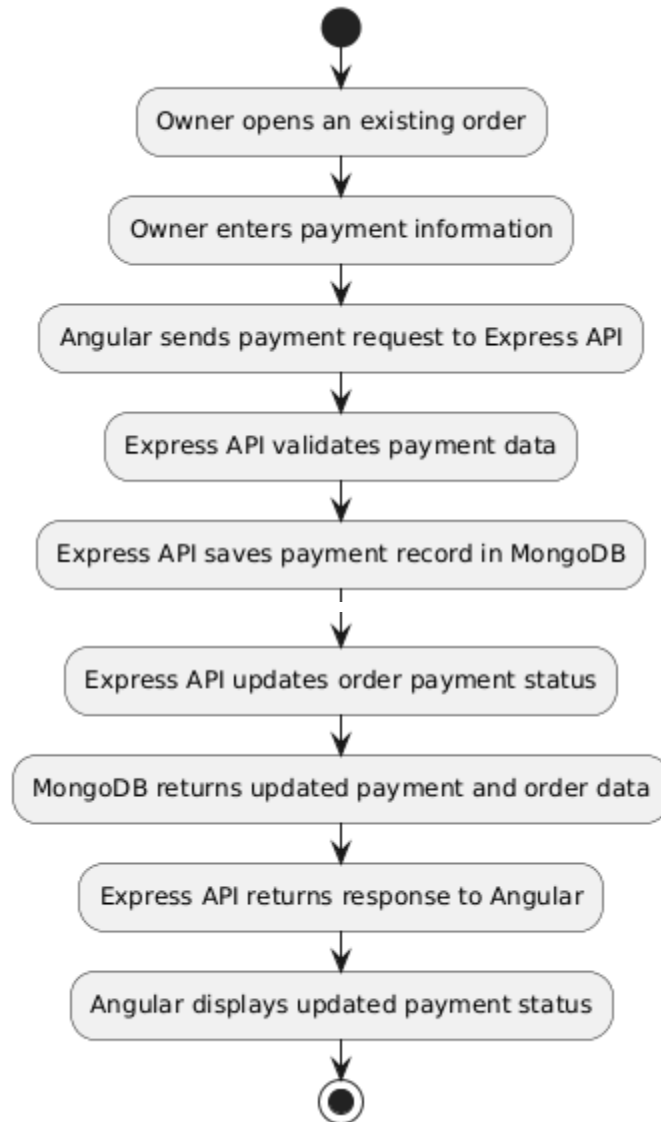


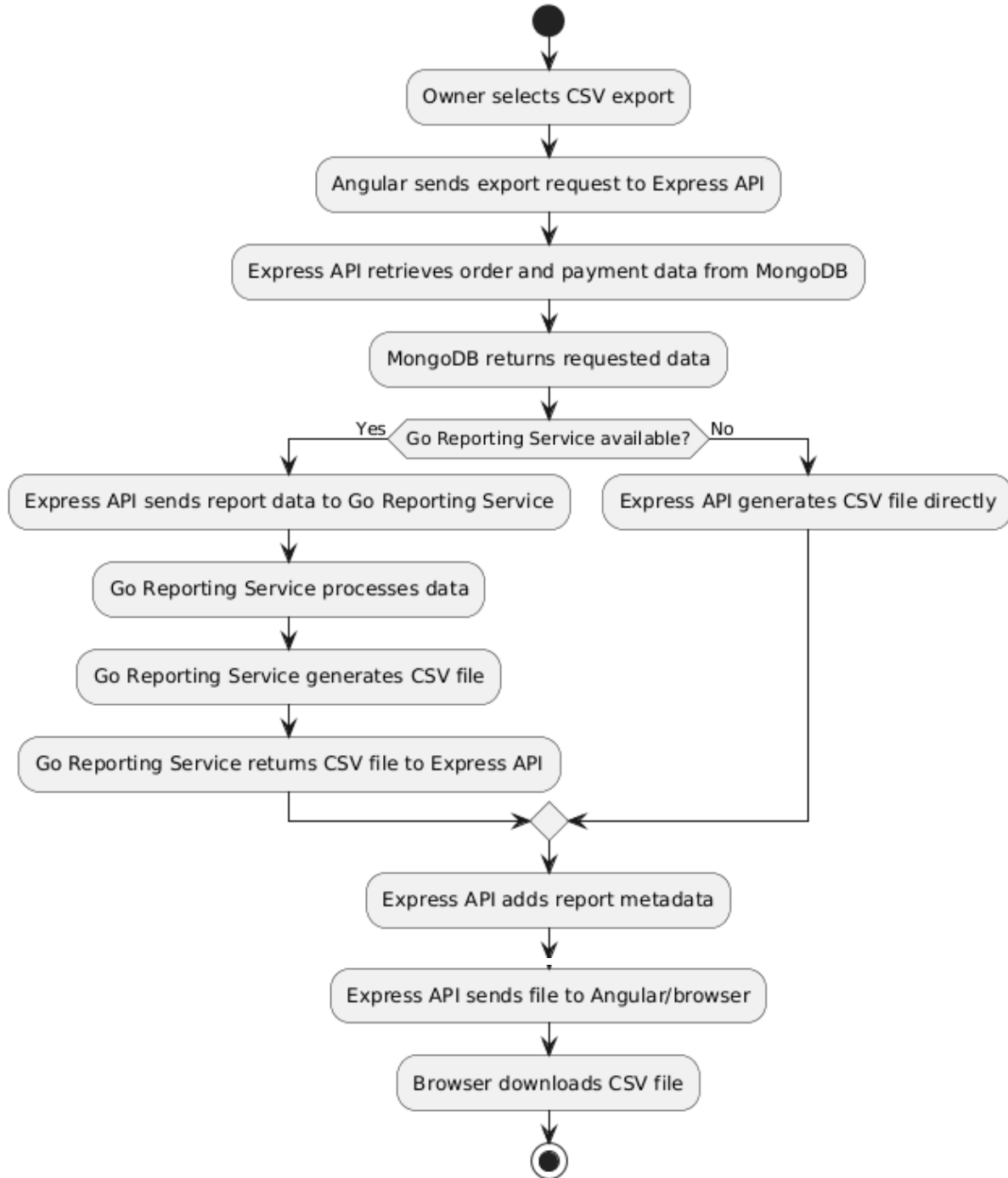
Workflow Phase 4 - Order Management





Workflow Phase 5 - Payment Tracking



Workflow Phase 6 - Reporting and CSV Export

The following wireframes illustrate the proposed Graphical User Interface (GUI) for the Home Gig application. The design focuses on providing a simple and efficient Human-Machine Interface (HMI) for small business owners while following Human-Computer Interaction (HCI) principles that support ease of use on both mobile and desktop devices.

The Home Gig user interface begins with a secure authentication process and is centered around a centralized owner dashboard that provides access to the application's primary business management functions. The dashboard allows business owners to manage products, inventory, customers, orders, payments, and reporting activities from a single location.

The wireframes presented in this section provide an initial prototype of the user interface and demonstrate the planned layout, navigation structure, and user workflows of the application. These wireframes represent the current UI/UX design and may be refined during implementation and stakeholder feedback activities.

Mobile & Desktop Figures Included:

- Login and Authentication
- Owner Dashboard
- Product and Inventory Management
- Customer Management
- Order Management
- Spreadsheet Export and Reporting
- Supporting Mobile Add/Edit Forms

UI/UX Wireframes

Together, these wireframes illustrate how users will navigate the system and perform the primary workflows identified in the requirements specification. The mobile wireframes are presented first because one of the primary goals of the Home Gig project is to provide convenient mobile access for small business owners

who frequently manage orders, customers, and payments while away from a desktop computer. The desktop wireframes provide the same functionality while taking advantage of additional screen space for data entry, management activities, and reporting tasks.

Note Regarding Wireframes

The wireframes presented in this section illustrate the planned user experience and overall interface design for the Home Gig application. To provide a complete view of the intended system, certain wireframe elements may represent stretch requirements or future enhancements that are not required for successful completion of the minimum viable product (MVP). The inclusion of a feature within a wireframe should not be interpreted as a commitment that the feature will be implemented as part of the project's required scope. Required functionality is defined by the Must Have Requirements listed in Section 2a, while additional functionality is governed by the Stretch Requirements listed in Section 2b.

Several management screens include supporting mobile forms for creating and editing records. To minimize navigation complexity on smaller devices, mobile users access dedicated Add and Edit views, while desktop users perform the same functions through integrated management screens.

The "Create Account," "Forgot Password," and related account management functionality are shown in the wireframes to illustrate the planned user experience. These elements are only available if the corresponding stretch requirements are implemented. In the MVP, user accounts will be created and managed by the system administrator, and password recovery functionality may not be available.

Figure 1M – Home Gig Login (Mobile)

The mobile login screen provides secure access to the Home Gig application from a mobile device. Users enter their email address and password to authenticate and access the application's business management features. The simplified mobile layout is designed to support quick access while maintaining usability on smaller screens.

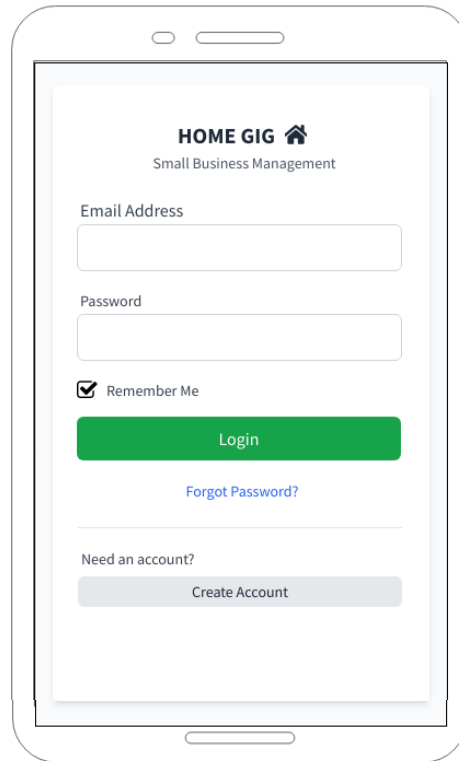
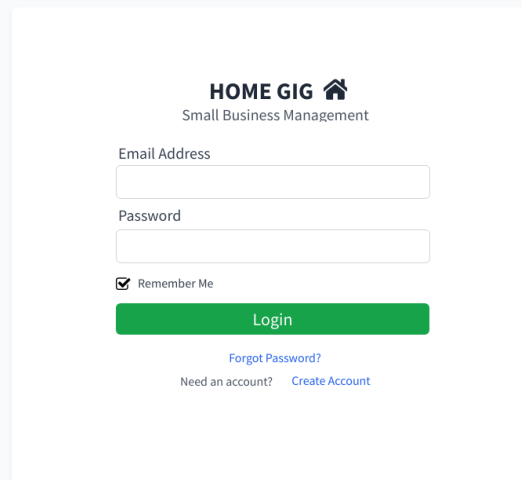


Figure 1D – Home Gig Login (Desktop)

The desktop login screen provides secure access to the Home Gig application from a desktop or laptop computer. Users authenticate using their email address and password before accessing the application's dashboard and management functions. The desktop layout provides a familiar authentication experience while maintaining consistency with the overall application design.



The image shows a login form for 'HOME GIG Small Business Management'. It features a header with the logo and name. Below are input fields for 'Email Address' and 'Password'. A 'Remember Me' checkbox is present. A green 'Login' button is centered. At the bottom, there are links for 'Forgot Password?', 'Need an account?', and 'Create Account'.

HOME GIG 
Small Business Management

Email Address

Password

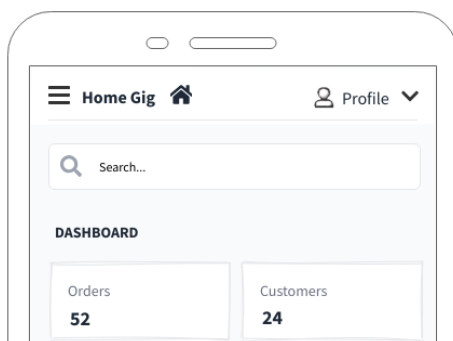
☒ Remember Me

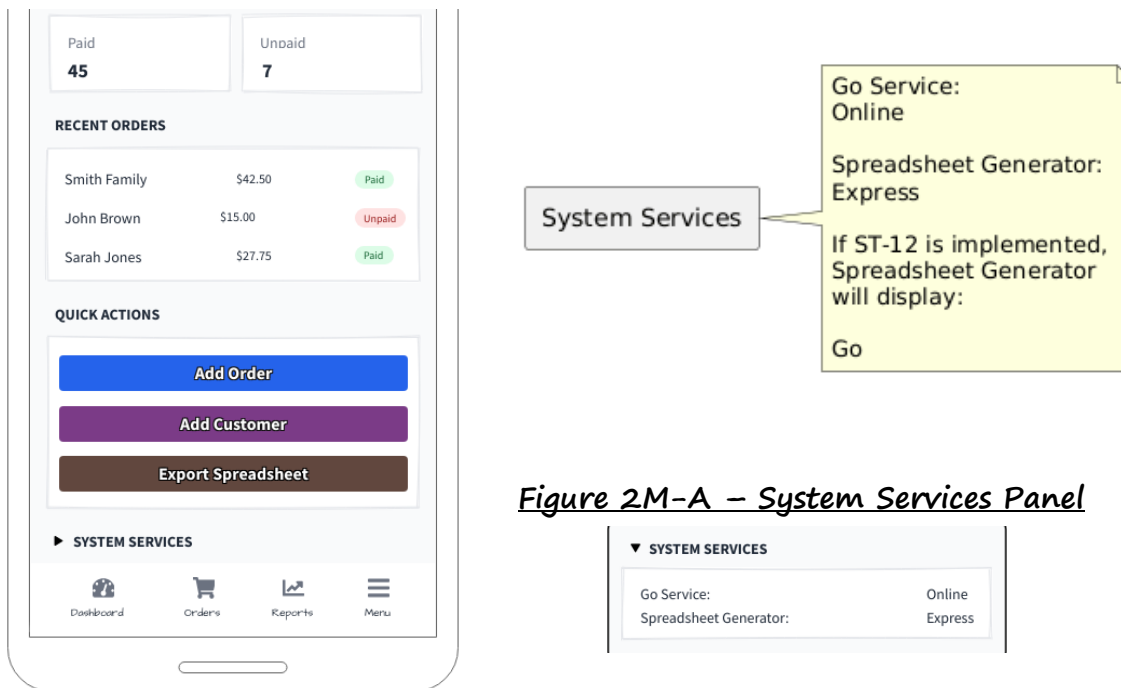
Login

[Forgot Password?](#)
[Need an account?](#) [Create Account](#)

Figure 2M - Home Gig Owner Dashboard (Mobile)

The mobile dashboard serves as the primary interface for the Home Gig application. Because one of the project's primary goals is to support business owners who manage their operations from a phone, the design emphasizes accessibility, simplicity, and efficient task completion on mobile devices. Users can view business metrics, manage orders and customers, track payments, monitor inventory, and generate reports without requiring access to a desktop computer.





(Expanded)

Figure 2M-A – System Services Panel

SYSTEM SERVICES	
Go Service:	Online
Spreadsheet Generator:	Express

Figure 2D – Home Gig Owner Dashboard (Desktop)

The desktop dashboard provides the same core functionality as the mobile interface while taking advantage of additional screen space for viewing larger data sets, performing management tasks, and working with reports. The desktop layout offers an expanded view of business information while maintaining a consistent user experience across devices.

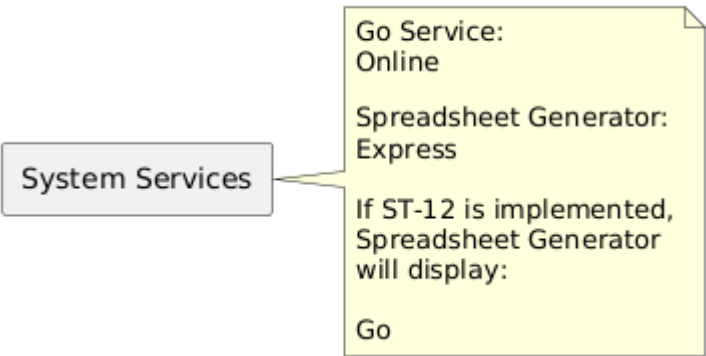
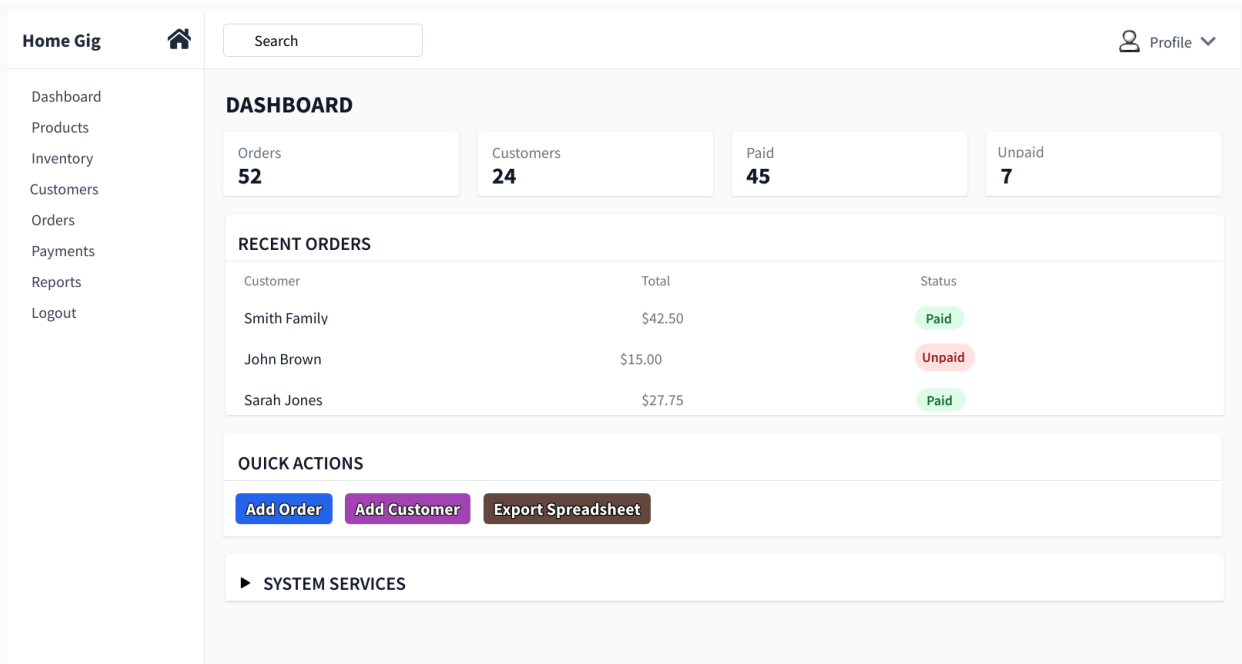


Figure 2D-A – System Services Panel (Expanded)

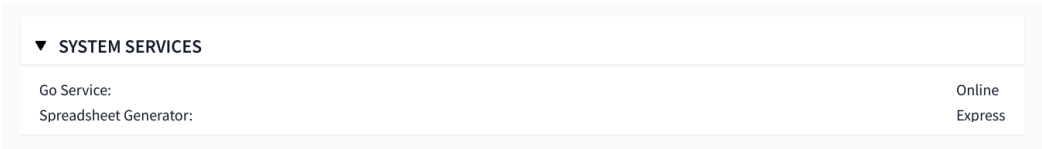


Figure 3M – Home Gig Product and Inventory Management (Mobile)

The mobile product and inventory management interface serves as a primary workflow for business owners who manage their operations from a mobile device. Inventory quantities are emphasized to support quick inventory monitoring, while product pricing and editing functions remain readily accessible. Low inventory

items are identified using warning indicators to help owners quickly identify products that may require replenishment.

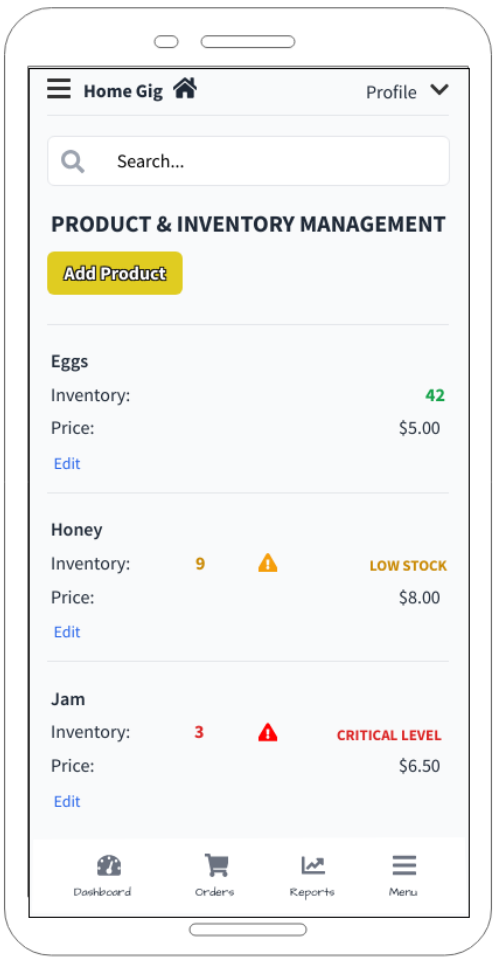
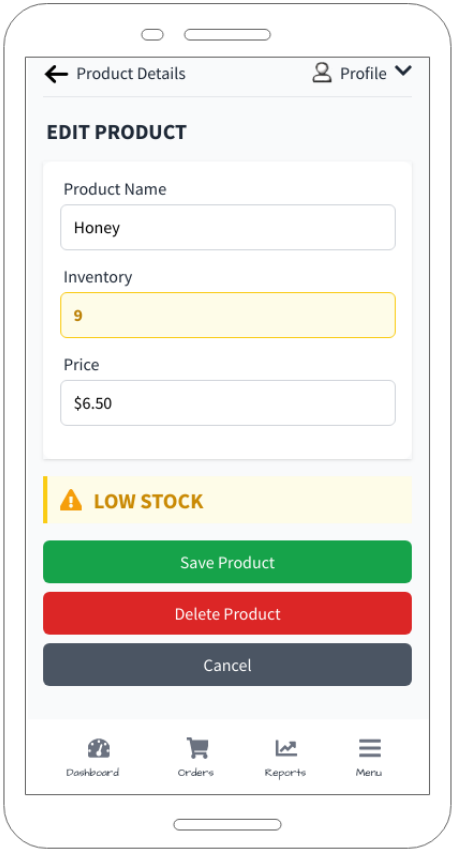


Figure 3M-A – Home Gig Edit Product View (Mobile)

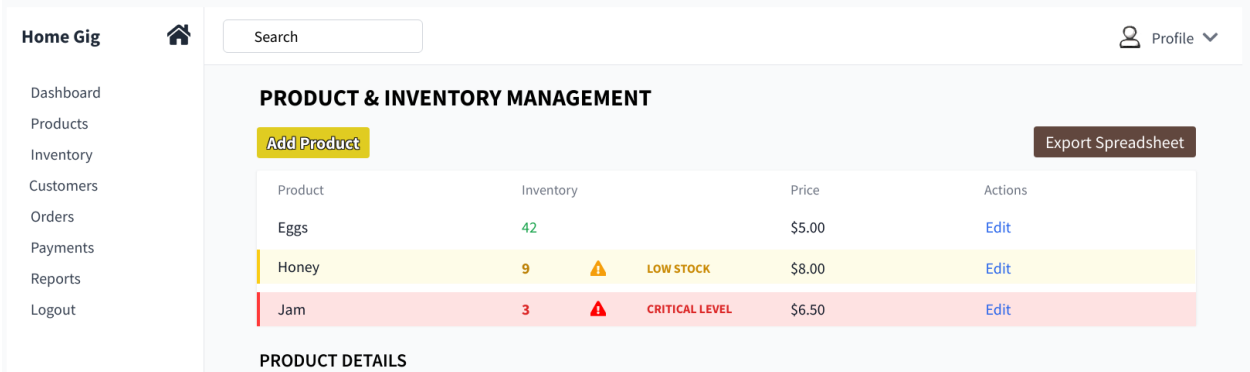
The mobile edit product view allows the business owner to update product name, inventory quantity, and price from a focused mobile screen.

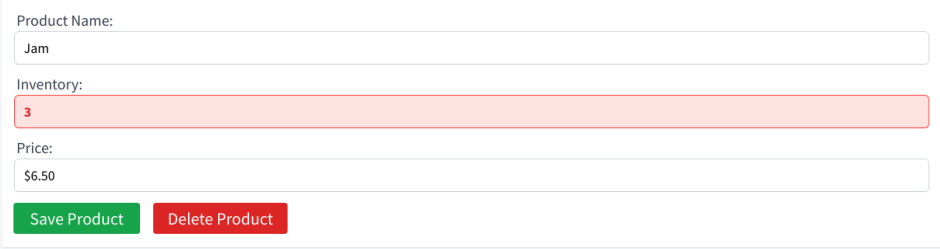


Selecting Add Product opens a blank product form. Selecting Edit loads the selected product into the same form for modification.

Figure 3D – Home Gig Product and Inventory Management (Desktop)

The desktop product and inventory management interface provides a centralized view of product information and inventory quantities. Inventory levels are prioritized within the product listing to help business owners quickly identify low-stock items while managing product information and pricing from a single screen.



A screenshot of a mobile application interface for product management. It features a form with three input fields: 'Product Name' containing 'Jam', 'Inventory' containing '3', and 'Price' containing '\$6.50'. The 'Inventory' field is highlighted with a red border. Below the fields are two buttons: a green 'Save Product' button and a red 'Delete Product' button.

Product Name:	Jam
Inventory:	3
Price:	\$6.50
Save Product Delete Product	

When the user selects *Edit* on a product or customer record, the selected record loads into the details form on the same page. The user can update the fields and select *Save* or *Delete* without leaving the management screen.

Figure 4M – Home Gig Customer Management (Mobile)

The mobile customer management interface allows business owners to create, view, edit, and manage customer information directly from a mobile device. Customer records are presented in a mobile-friendly format that provides quick access to contact information while supporting efficient customer management activities from any location.

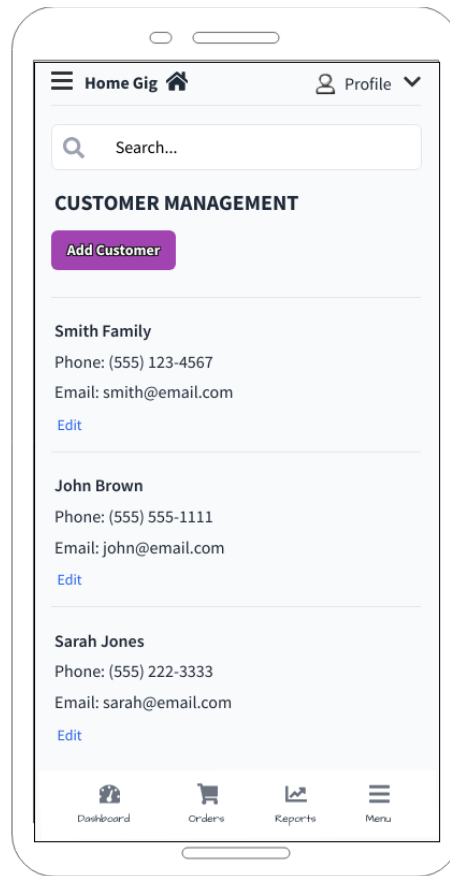
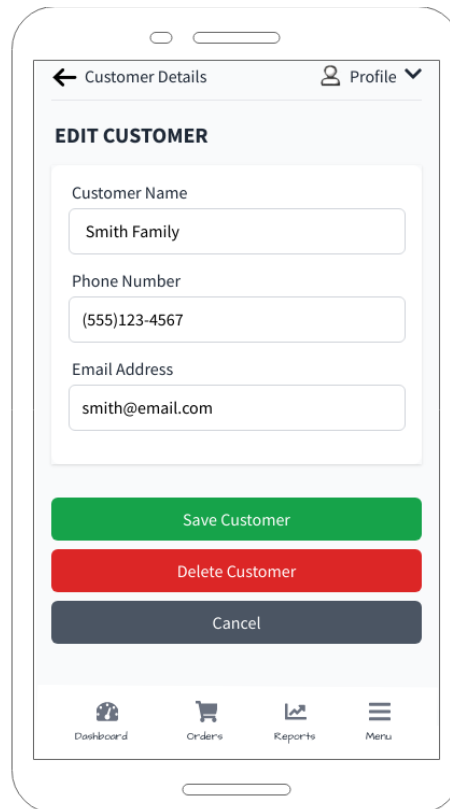


Figure 4M-A – Home Gig Edit Customer View (Mobile)

The mobile edit customer view allows the business owner to update customer contact information from a focused mobile screen.



The image shows a mobile application interface for editing a customer. At the top, there is a header bar with a back arrow, the text 'Customer Details', and a 'Profile' dropdown menu. Below this is a section titled 'EDIT CUSTOMER'. Inside this section, there are three input fields: 'Customer Name' with the value 'Smith Family', 'Phone Number' with the value '(555)123-4567', and 'Email Address' with the value 'smith@email.com'. Below the input fields are three buttons: a green 'Save Customer' button, a red 'Delete Customer' button, and a grey 'Cancel' button. At the bottom of the screen is a navigation bar with four icons and labels: 'Dashboard', 'Orders', 'Reports', and 'Menu'.

Selecting Add Customer opens a blank customer form. Selecting Edit loads the selected customer into the same form for modification.

Figure 4D – Home Gig Customer Management (Desktop)

The desktop customer management interface provides a centralized view of customer information and management functions. Business owners can create, edit, and maintain customer records while viewing contact information from a single screen.

Home Gig

Dashboard

Products

Inventory

Customers

Orders

Payments

Reports

Logout

Search

Profile

CUSTOMER MANAGEMENT

Add Customer

Export Spreadsheet

Customer	Phone Number	Email	Actions
Smith Family	(555)123-4567	smith@email.com	Edit
John Brown	(555)555-1111	john@email.com	Edit
Sarah Jones	(555)222-3333	sarah@email.com	Edit

CUSTOMER DETAILS

Customer Name:

Smith Family

Phone Number:

(555)123-4567

Email Address:

smith@email.com

Save Customer

Delete Customer

When the user selects *Edit* on a customer record, the selected record loads into the details form on the same page. The user can update the fields and select *Save* or *Delete* without leaving the management screen.

Figure 5M – Home Gig Order Management (Mobile)

The mobile order management interface allows business owners to create, view, edit, and manage customer orders directly from a mobile device. Orders display customer information, ordered products, quantities, and payment status to support efficient order tracking while away from a desktop computer.

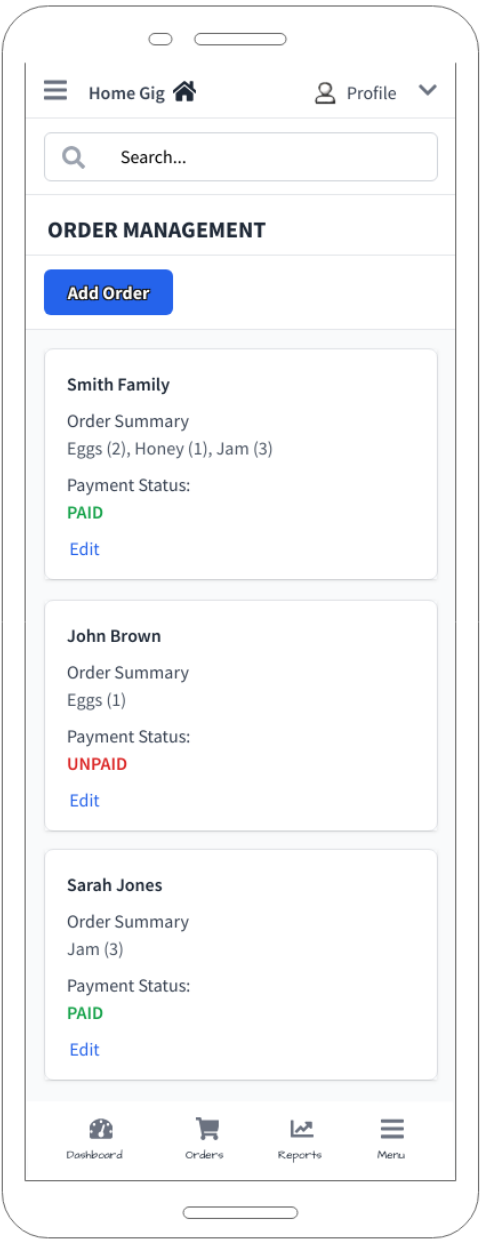


Figure SM-A – Home Gig Edit Order View (Mobile)

The mobile edit order view allows the business owner to modify order information, update product quantities, and record payment status from a focused mobile screen.

The screenshot shows a mobile application interface for managing orders. At the top, there's a navigation bar with a back arrow, 'Order Details', and a profile icon with a dropdown arrow. Below this is a section titled 'EDIT ORDER'. Inside this section, there's a form with the following elements:

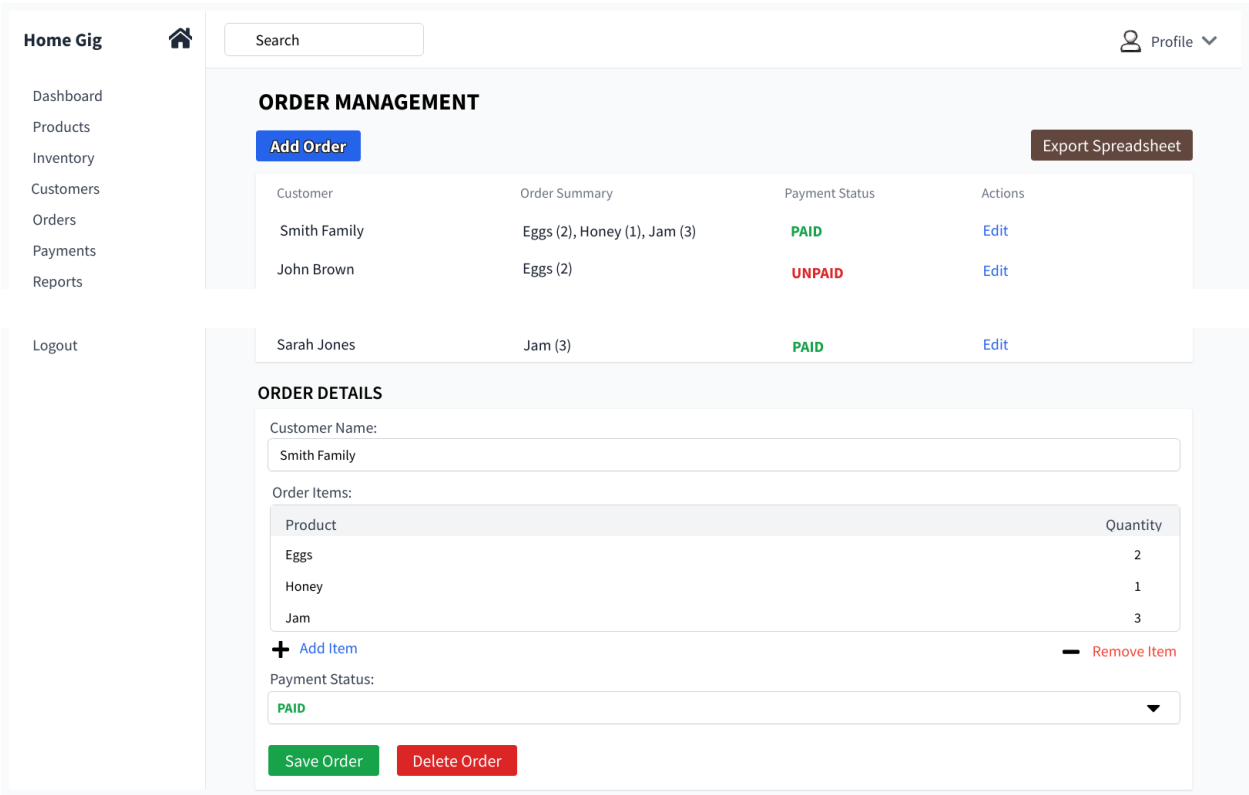
- Customer Name:** A text input field containing 'Smith Family'.
- Order Items:** A table with two columns: 'Product' and 'Quantity'.

Product	Quantity
Eggs	2
Honey	1
Jam	3
- Buttons:** Below the table are two buttons: '+ Add Item' (blue) and '- Remove Item' (red).
- Payment Status:** A dropdown menu currently showing 'PAID' in green text.
- Action Buttons:** At the bottom of the form are three large buttons: 'Save Order' (green), 'Delete Order' (red), and 'Cancel' (dark grey).
- Bottom Navigation Bar:** Contains four icons with labels: 'Dashboard' (home icon), 'Orders' (shopping cart icon), 'Reports' (line graph icon), and 'Menu' (hamburger menu icon).

Selecting Add Order opens a blank order form. Selecting Edit loads the selected order into the same form for modification.

Figure 5D – Home Gig Order Management (Desktop)

The desktop order management interface provides a centralized view of customer orders and payment status information. Business owners can create, edit, and manage orders while viewing customer, product, quantity, and payment information from a single screen.



When the user selects *Edit* on an order record, the selected record loads into the details form on the same page. The user can update the fields and select *Save* or *Delete* without leaving the management screen.

Figure 6M – Home Gig Spreadsheet Export & Reporting (Mobile)

The mobile spreadsheet export interface allows business owners to generate and download spreadsheet-compatible reports directly from a mobile device. Users may select report options, spreadsheet format, and date ranges before generating exports containing order and payment information for accounting and record-keeping purposes.

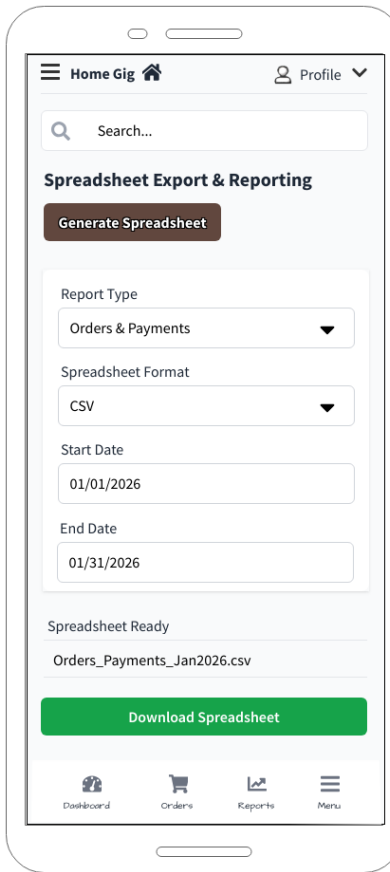


Figure 6D – Home Gig Spreadsheet Export & Reporting (Desktop)

The desktop spreadsheet export interface provides the same export functionality while taking advantage of additional screen space for report generation and file management activities. Users may generate and download spreadsheet-compatible reports containing order and payment information based on selected export options and date ranges.

Home Gig   Profile 

- Dashboard
- Products
- Inventory
- Customers
- Orders
- Payments
- Reports
- Logout

SPREADSHEET EXPORT & REPORTING

Generate Spreadsheet

Report Type:

Spreadsheet Format:

Start Date:

End Date:

GENERATED SPREADSHEET

Results:

Spreadsheet Ready

Orders_Payments_Jan2026.csv

Download Spreadsheet

The spreadsheet format selection shown in these wireframes represents planned extensibility for future spreadsheet export enhancements. CSV export is required for the MVP, while support for additional spreadsheet formats may be implemented as stretch requirements.

Section 4: Verification

Verification Approach

Verification activities will be performed throughout the development process to ensure that Home Gig satisfies its functional and non-functional requirements. Verification will include developer testing, integration testing, user acceptance testing (UAT), and demonstration-based validation with project stakeholders.

The primary verification objective is to demonstrate that business owners can successfully manage products, inventory, customers, orders, payments, and reporting activities using the Home Gig application. Each requirement will be

verified through a combination of functional testing, workflow demonstrations, and acceptance testing using realistic business scenarios.

Testing will be conducted using representative sample data including products, inventory quantities, customers, orders, and payments. Test data will be designed to simulate the daily operations of a small home-based business. If Stretch Requirement ST-16 is implemented, additional delivery scheduling and tracking data may also be included in testing activities.

Demonstration Plan

The final project demonstration will validate that the application functions according to the approved requirements and design specifications. Demonstration activities will include:

- User authentication and login.
- Product and inventory management.
- Customer record management.
- Order creation and management.
- Manual order entry for verbal or phone orders.
- Payment tracking and status updates.
- Inventory quantity adjustments.
- Spreadsheet export generation.
- Mobile and desktop interface demonstrations.

Sample business data will be created before the demonstration, including:

- Multiple products with inventory quantities.
- Multiple customer records.
- Several orders in various processing states.
- Payment records with different statuses.

➤ Historical order information for reporting and export testing.

Successful completion of the demonstration will be verified by executing predefined business workflows and confirming that the expected results are produced.

Testing Approach

Testing activities will include unit testing, integration testing, system testing, and user acceptance testing.

Unit Testing

Individual frontend and backend functions will be tested to verify that they perform their intended operations and properly handle valid and invalid inputs.

Integration Testing

Interactions between the Angular frontend, Express API, and MongoDB database will be tested to ensure data is correctly transmitted, processed, stored, and retrieved.

System Testing

End-to-end workflows will be executed to verify that complete business processes function correctly from user input through database persistence and reporting.

User Acceptance Testing

Stakeholders will evaluate the application using realistic business scenarios to confirm that the system meets operational needs and provides acceptable usability.

Requirement Verification

Testing activities will include unit testing, integration testing, system testing, and user acceptance testing.

MH-1: User Authentication Verification

Verification Method:

Test successful login, failed login, password protection, authenticated API access, and unauthenticated API access.

Pass Criteria:

- ✓ A valid user can log in successfully.
- ✓ Invalid credentials are rejected.
- ✓ Passwords are not stored in plain text.
- ✓ Protected API endpoints reject unauthenticated requests.
- ✓ Protected API endpoints allow access after valid authentication.

Fail Criteria:

- ✗ Valid credentials cannot log in.
 - ✗ Invalid credentials are accepted.
 - ✗ Valid credentials are rejected.
 - ✗ Authentication errors occur.
-

MH-2: Product Management Verification

Verification Method:

Create, view, update, and delete product records.

Pass Criteria:

- ✓ Product information is successfully saved.
- ✓ Product changes persist after application restart.
- ✓ Deleted products are no longer displayed in active product lists.

Fail Criteria:

- ✗ Product information is not saved.
 - ✗ Updates are lost.
 - ✗ Passwords are stored in plain text.
 - ✗ Protected API endpoints allow unauthenticated access.
 - ✗ Authenticated users cannot access protected API endpoints.
-

MH-3: Inventory Management Verification

Verification Method:

Create products with inventory quantities and perform inventory updates through manual adjustments and order processing.

Pass Criteria:

- ✓ Inventory quantities update correctly.
- ✓ Inventory changes are stored in the database.
- ✓ Inventory values remain accurate after application restart.

Fail Criteria:

- ✗ Inventory quantities become inaccurate.
- ✗ Inventory updates are not persisted.
- ✗ Inventory values do not reflect business activity.

MH-4: Centralized Order Management Verification

Verification Method:

Create, modify, view, and complete orders using sample customer and product data.

Pass Criteria:

- ✓ Orders are successfully created and stored.
- ✓ Order status changes are retained.
- ✓ Order information is accurately displayed and retrieved.

Fail Criteria:

- ✗ Orders are not saved.
- ✗ Status changes are lost.
- ✗ Order information becomes inconsistent.

MH-5: Customer Management Verification

Verification Method:

Create, view, update, and delete customer records.

Pass Criteria:

- ✓ Customer information is successfully saved and retrieved.
- ✓ Updates are reflected throughout the application.
- ✓ Deleted customer records are removed from active lists.

Fail Criteria:

- ✗ Customer data is not saved.
 - ✗ Updates are not reflected.
 - ✗ Deleted records remain available.
-

MH-6: Payment Tracking Verification

Verification Method:

Create orders with associated payment records and update payment status values.

Pass Criteria:

- ✓ Payment information is stored correctly.
- ✓ Payment status updates are retained.
- ✓ Payment records remain associated with the correct order.

Fail Criteria:

- ✗ Payment information is lost.
 - ✗ Status updates are not saved.
 - ✗ Payments become disconnected from associated orders.
-

MH-7: CSV Export Verification

Verification Method:

Generate CSV exports using existing product, customer, order, inventory, and payment data.

Pass Criteria:

- ✓ CSV files are successfully generated and downloaded.
- ✓ Exported data matches database records.
- ✓ Files can be opened using spreadsheet software.

- ✓ Generated files contain report metadata including the report generation service and generation date/time.

Fail Criteria:

- ✗ CSV files fail to generate.
 - ✗ Exported data is incomplete or inaccurate.
 - ✗ Files cannot be opened successfully.
 - ✗ Required report metadata is missing or incorrect.
-

MH-8: Go System Status Service Verification

Verification Method:

Start the Go Service and access the system status endpoint through the Home Gig application.

Pass Criteria:

- ✓ The Go Service starts successfully.
- ✓ The status endpoint returns a valid response.
- ✓ The Home Gig application successfully communicates with the Go Service.
- ✓ The System Services panel displays "Go Service: Online".

Fail Criteria:

- ✗ The Go Service fails to start.
 - ✗ The status endpoint does not return a valid response.
 - ✗ Communication between the Home Gig application and the Go Service fails.
 - ✗ The System Services panel does not correctly display the service status.
-

Sources / Citations / Resource Links

The following sources are some resources that were used during requirements gathering, technology selection, architecture planning, and user interface design for the Home Gig project. Additional implementation resources are also included because they are expected to support development of planned and stretch functionality. Requirements were derived primarily from stakeholder discussions

with a small home-based business owner and supplemented by technology documentation, industry best practices, and software engineering resources used during project planning.

Stakeholder Sources

1. Home Gig Business Owner Stakeholder discussions.
 - Informal requirements gathering discussions regarding order management, customer tracking, inventory management, payment tracking, and spreadsheet export needs for small home-based businesses.

Technology Documentation

2. Angular Documentation
 - <https://v17.angular.io/docs>
3. What is Angular
 - <https://angular.dev/overview>
4. Node.js Documentation
 - <https://nodejs.org/docs/latest/api/>
5. Learn Node.js
 - <https://nodejs.org/learn>
6. Express.js Documentation
 - <https://expressjs.com/en/>
7. Go Documentation
 - <https://go.dev/doc/>
8. Golang Project Layout Guide
 - <https://github.com/golang-standards/project-layout>
9. Tutorial: Get started with Go
 - <https://go.dev/doc/tutorial/getting-started>
10. MongoDB Documentation
 - <https://www.mongodb.com/docs>
11. Mongoose Documentation
 - <https://mongoosejs.com/docs>
12. Render

- <https://render.com/>

Spreadsheet Export Resources

13. CSV File Format Reference
 - <https://www.rfc-editor.org/info/rfc4180/>
14. ExcelJS Documentation
 - <https://github.com/exceljs/exceljs>
15. SheetJS Community Edition
 - <https://docs.sheetjs.com/>
16. Building spreadsheets with Go and Excelize
 - <https://blog.logrocket.com/building-spreadsheets-go-excelize/>
17. Processing Spreadsheet Data in Go
 - <https://appliedgo.net/spreadsheet/>
18. Excelize documentation and GitHub repo
 - <https://github.com/qax-os/excelize>
19. A Guide to Working with Excel Files in Golang - Excelize
 - <https://www.kelche.co/blog/go/excel/>

Progressive Web Application (PWA) Resources

20. Angular Service Worker & PWA Documentation
 - <https://angular.dev/ecosystem/service-workers>
21. NPM @angular/pwa
 - <https://www.npmjs.com/package/@angular/pwa>
22. Progressive Web Apps in Angular
 - <https://ionicframework.com/docs/angular/pwa>
23. Creating a Progressive Web Application with Angular
 - <https://dontpaniclabs.com/blog/post/2019/01/02/creating-a-progressive-web-application-with-angular/>
24. Building Your First Angular PWA with Ignite UI for Angular
 - <https://www.infragistics.com/blogs/angular-pwa>
25. Progressive Web Apps (PWA) Overview
 - <https://web.dev/explore/progressive-web-apps>

26. Web App Manifest Documentation

- [https://developer.mozilla.org/en-US/docs/Web/Progressive web apps/Manifest](https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Manifest)

User Interface and Design Resources

27. MockFlow

- <https://app.mockflow.com/>

28. OpenAI ChatGPT

- <https://chatgpt.com/>

29. Magnific (Image Resources)

- <https://www.magnific.com/>

30. Vecteezy (Image Resources)

- <https://www.vecteezy.com/>

31. Icons8 (Image Resources)

- <https://icons8.com/>

32. Material Design

- <https://m3.material.io/>

33. Responsive Web Design Basics

- <https://web.dev/articles/responsive-web-design-basics>

34. Research-based, Results-driven UX Guidance and Training

- <https://www.nngroup.com/>

35. Get started with Bootstrap

- <https://getbootstrap.com/docs/5.3/getting-started/introduction/>

36. Bootstrap 5 Tutorial

- <https://www.w3schools.com/bootstrap5/>

Development and Planning Resources

37. Visual Studio Code

- <https://code.visualstudio.com/>

38. Git Documentation

- <https://git-scm.com/docs>

39. GitHub Documentation

- <https://docs.github.com/en>
40. BYU-Idaho Course Materials
- WDD 430 Full-Stack Web Development
 - CSE 499A Senior Project

AI-Assisted Development Resources

41. OpenAI ChatGPT
- <https://chatgpt.com/>
42. Codex
- <https://developers.openai.com/codex>
43. SuperGrok
- <https://grok.com/>
44. Claude
- <https://claude.com/>
45. Perplexity
- <https://www.perplexity.ai/>
46. Cursor
- <https://cursor.com/students>
47. Superpowers Skill
- <https://github.com/obra/superpowers>
 - <https://www.superpowers-skills.com/en-us/index.html>
48. Tips for building with GitHub Copilot
- <https://github.com/features/copilot/tutorials>